

Documento de Diseño Técnico y Funcional (Conceptual)

Sistema de Gestión de Habitaciones y Operaciones (GeHO)

Fase del Proyecto	Fase 2: Diseño de Arquitectura y Componentes	Fecha de Entrega	18 de Mayo de 2026
Autores (Desarrollo)	Alejandro Martínez Bernal (No. Control: 22100209) — <i>DevOps Lead</i> Jorge Arturo Mata Camacho (No. Control: C21100514) — <i>Backend Developer</i>	Colaboradores (DBA & UI)	Christhofer Solis Zamarron (DBA) Dana Jael Garcia Martinez (Frontend)

1. Arquitectura General del Sistema

El Sistema GeHO ha sido rediseñado bajo un enfoque arquitectónico desacoplado y moderno mediante el uso de contenedores automatizados con Docker. Esta topología de infraestructura garantiza que el ecosistema completo de software pueda ejecutarse de forma idéntica en cualquier entorno operativo de desarrollo o producción, eliminando los conflictos clásicos de dependencias locales y agilizando el flujo de integración continua.

La topología estructural y el intercambio de información entre las capas del sistema se describe visualmente en el siguiente diagrama técnico de arquitectura conceptual:

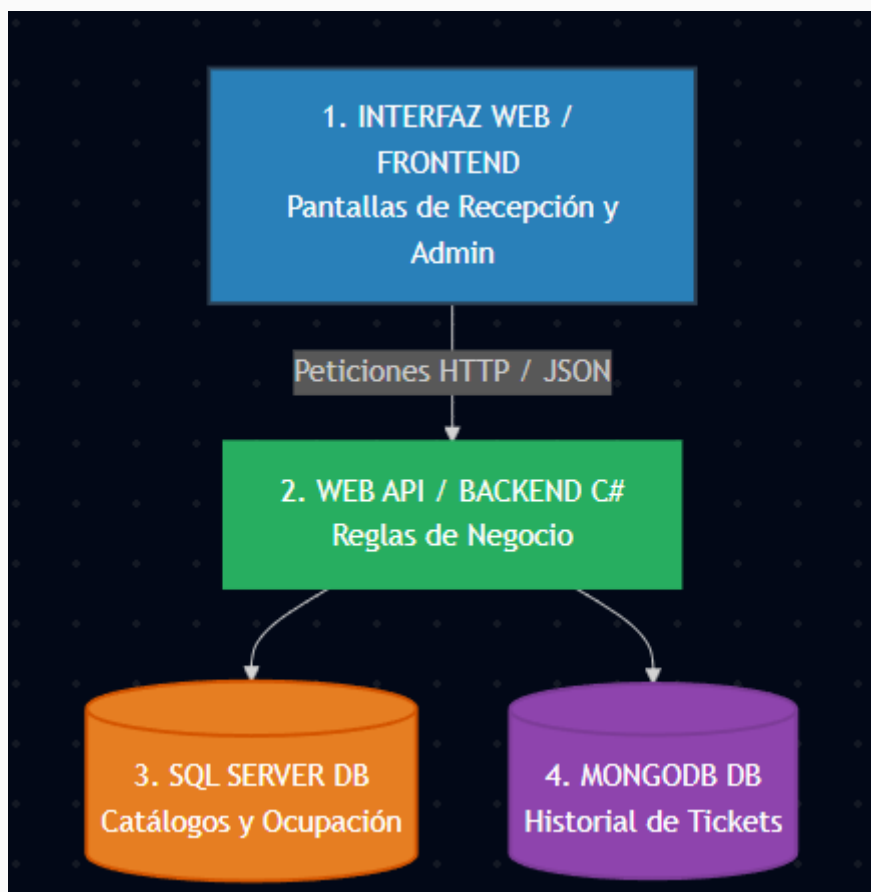


Figura 1.1: Diagrama de bloques y flujo de peticiones estructuradas en el entorno híbrido de GeHO.

2. Descripción de los Componentes

El flujo operativo se orquesta a través de tres componentes especializados distribuidos lógicamente según su responsabilidad dentro del sistema:

2.1. Interfaz Web / Frontend (Coordinado por Dana)

Representa la capa de interacción directa con el usuario (receptionistas y administradores). Es una aplicación web reactiva que consume de manera asíncrona los servicios expuestos por el Backend. Su responsabilidad principal es procesar las interacciones de la UI y estructurar los payloads dinámicos en formato JSON para mandarlos a la API.

2.2. Web API / Backend en C# (Desarrollado por Jorge)

Construido sobre ASP.NET Core 8, actúa como el cerebro del sistema y la única puerta de entrada hacia las bases de datos. Aplica de manera rigurosa las reglas de negocio del complejo hotelero (validación de estados, cálculo de tarifas, autenticación basada en roles) y despacha las transacciones concurrentes de forma segura.

2.3. Infraestructura y Contenedores / DevOps (Liderado por Alejandro)

Gestionado de manera centralizada a través de un archivo de orquestación de Docker Compose. Este componente encapsula de forma aislada la API del backend y ambos motores de bases de datos dentro de una red virtual interna segura, garantizando portabilidad absoluta y despliegues automáticos limpios.

3. Estrategia de Persistencia Híbrida (Bases de Datos)

Con el fin de maximizar el rendimiento, garantizar la inmutabilidad financiera y facilitar las auditorías solicitadas por la dirección, el sistema adopta una estrategia de persistencia dual o híbrida coordinada en conjunto con el DBA (Chris):

Motor de Persistencia	Responsabilidad y Criterio de Uso Técnico
SQL Server BASE DE DATOS RELACIONAL	Soporta la "operación viva" del hotel. Gestiona datos estructurados y relaciones complejas que requieren integridad referencial estricta y transacciones ACID. Almacena el catálogo de habitaciones, el estado actual en tiempo real de los cuartos (Disponible, Ocupada, Sucia), inventario de la tienda y las credenciales de los usuarios con acceso al sistema.
MongoDB BASE DE DATOS DOCUMENTAL	Se utiliza de manera exclusiva como un repositorio histórico e inmutable de Tickets (tanto cierres de cuenta de hospedaje como notas de venta de la tienda). Cada venta genera un documento JSON estático e independiente. Esto soluciona de forma definitiva el error del sistema anterior: si un administrador modifica el precio de un cuarto o producto hoy en SQL Server, los comprobantes históricos en MongoDB no se alteran, blindando la contabilidad.

4. Flujo Técnico de Módulos Clave

La intercomunicación funcional entre la interfaz, la lógica de negocio en C# y los motores híbridos de datos se valida a través de tres flujos fundamentales:

- **Dashboard de Habitaciones (Monitoreo Dinámico):** El Frontend efectúa una petición de lectura asíncrona hacia el Backend. El Backend consulta de forma directa la tabla relacional de habitaciones en SQL Server y retorna la colección en formato JSON. El Frontend mapea este arreglo para renderizar la cuadrícula de pisos con sus respectivos colores técnicos (Verde para Disponible, Rojo para Ocupado, Azul para Limpieza).

- **Proceso Automatizado de Check-out (Salida):** Al liquidar una estancia, el recepcionista envía la orden al Backend. La Web API realiza dos operaciones atómicas simultáneas: actualiza el estado de la habitación a "Limpieza" en SQL Server para alertar al personal de mantenimiento, y congela toda la información financiera y del huésped en un documento JSON autónomo que se escribe en la colección histórica de MongoDB.
- **Punto de Venta Integrado (Tienda):** Al confirmar un carrito de compras con productos consumidos por el huésped, el Backend descuenta de forma inmediata el inventario del stock real en SQL Server (operación relacional) e inyecta la nota de consumo final con los precios exactos del momento en el repositorio NoSQL de MongoDB.